

TALLER PRÁCTICO – BLOQUE 5

Integrantes: Karen Valentina Arteaga - Mario Zambrano

Actividad 1 – Diagnóstico de relaciones

Elementos	¿Qué relación existe en el problema?	¿Existencia independiente?	Decisión inicial
Estudiante – Préstamo	Un estudiante realiza préstamos; el préstamo registra a quién se le prestó el ejemplar. Son conceptos distintos que se conocen y colaboran.	Sí. El estudiante existe antes, durante y después de sus préstamos; un préstamo no “contiene” al estudiante.	Asociación
Libro – Ejemplar	R10: un libro tiene varios ejemplares físicos y cada ejemplar corresponde a un único libro. Hay una relación todo–parte, pero el enunciado no dice qué pasa con los ejemplares si el libro se retira.	Parcial / depende de un supuesto: el ejemplar es una copia de un título concreto y no se reasigna a otro libro.	Composición (con supuesto explícito, ver 2.2)
Préstamo – Renovación	R11: el préstamo registra sus renovaciones (fecha anterior, nueva fecha y fecha de la renovación). Son el historial interno del préstamo.	No. Una renovación solo tiene sentido dentro del préstamo que la originó.	Composición
Usuario – Estudiante	R12: el estudiante comparte identificación, nombre y correo con los demás usuarios y añade datos propios. El estudiante es una clase de usuario de la biblioteca.	No aplica (no es todo–parte): es una relación de especialización.	Candidato a generalización (se evalúa en la Sección 3)
Usuario – Bibliotecario	R12: igual que el estudiante; añade código de empleado y turno.	No aplica (no es todo–parte).	Candidato a generalización (se evalúa en la Sección 3)

Preguntas de discusión

1. ¿Que dos clases se relacionan significa necesariamente que una sea parte de la otra?

No. Dos clases pueden relacionarse solo porque se conocen o colaboran (asociación). Para hablar de todo–parte se necesita además evidencia de que una es componente conceptual de la otra y de cómo se comporta su ciclo de vida. Estudiante y Préstamo se relacionan, pero el estudiante no es parte del préstamo.

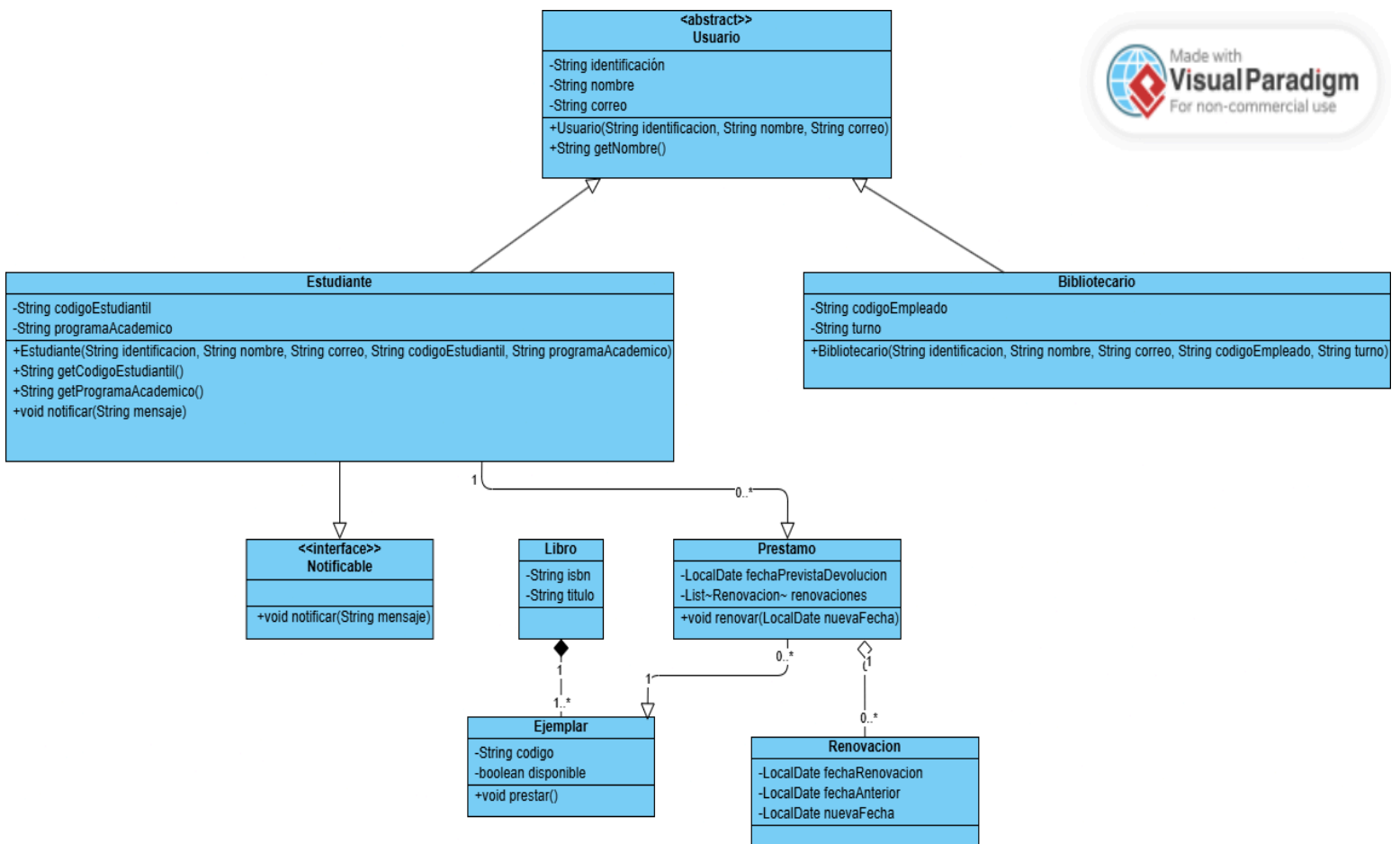
2. ¿Qué evidencia del requisito utilizaron?

R10 (“cada ejemplar corresponde a un único libro”) para fijar la multiplicidad y la dependencia del ejemplar; R11 (“un préstamo registra las renovaciones”) para ubicar el historial dentro del préstamo; R12 (“comparten identificación, nombre y correo, pero poseen información particular”) para plantear la generalización; R13 (“algunos usuarios”) para decidir que Notificable no recae sobre todo Usuario.

3. ¿Cuál sería incorrecto decidir mirando solo los nombres de las clases?

Libro–Ejemplar. El nombre sugiere “un libro tiene ejemplares” (¿agregación o composición?), pero la decisión depende de una regla de ciclo de vida que no aparece en el nombre y que hay que declarar como supuesto. También sería engañoso decidir Estudiante–Préstamo por nombre: “el préstamo tiene un estudiante” suena a contención, y no lo es.

Actividad 2 – Asociación, agregación y composición



Relación	Tipo seleccionado	Justificación basada en el dominio
Estudiante – Prestamo	Asociación	Solo se necesita expresar que ambos objetos se conocen: el préstamo sabe a qué estudiante se otorgó. No hay relación todo–parte. El estudiante tiene sentido y ciclo de vida independientes: eliminar un préstamo no elimina al estudiante.
Prestamo – Ejemplar	Asociación	Un préstamo recae sobre exactamente un ejemplar físico; con el tiempo un ejemplar acumula varios préstamos (histórico). El ejemplar existe antes y después de cualquier préstamo. Supuesto: un ejemplar solo puede tener un préstamo activo a la vez (regla que se validará en un servicio posterior; no se implementa en este bloque).
Libro – Ejemplar	Composición	R10: “cada ejemplar corresponde a un único libro”. Supuesto que valida la decisión: el ejemplar es el registro de inventario de una copia de un título concreto; no se reasigna a otro libro y, si el título se da de baja del catálogo, sus ejemplares dejan de tener significado en SmartLibrary. Por eso solo Libro crea ejemplares (agregarEjemplar). Se admite 0 ejemplares porque un título puede registrarse antes de recibir copias. Si el supuesto cambiara (ejemplares que sobreviven al libro o se reasignan), la relación pasaría a agregación.
Prestamo – Renovacion	Composición	R11: la renovación es parte del historial interno del préstamo. Sus tres fechas solo se interpretan dentro de ese préstamo, carece de sentido independiente y su ciclo de vida lo controla el préstamo (lo crea, lo almacena y expone solo una vista de lectura). Si el préstamo desaparece conceptualmente, sus renovaciones también.

Criterios aplicados (como pregunta, no como fórmula)

Pregunta	Resultado en SmartLibrary
¿Solo necesito expresar que los objetos se conocen o colaboran?	Sí en Estudiante–Prestamo y Prestamo–Ejemplar → asociación.
¿Existe relación conceptual todo–parte?	Sí en Libro–Ejemplar y Prestamo–Renovacion.
¿La parte tiene sentido y ciclo de vida independiente del todo?	No en Renovacion; no en Ejemplar bajo el supuesto declarado.
¿Eliminar conceptualmente el todo hace que la parte pierda significado?	Sí en ambos casos → composición.

Reto de razonamiento. El enunciado deja abierta la regla de ciclo de vida de Libro–Ejemplar. Otro equipo podría proponer agregación suponiendo que los ejemplares físicos subsisten aunque el título salga del catálogo (por ejemplo, para reasignarlos). Ambas posturas son válidas si el supuesto está escrito; aquí se eligió composición con el supuesto de la tabla anterior.

Actividad 3 - Herencia: ¿es-un o solo se parecen?

1. **¿Estudiante es realmente un tipo de Usuario?** Sí. R12 habla de “usuarios de biblioteca” y dice que estudiantes y bibliotecarios lo son. Un Estudiante puede usarse en cualquier contexto donde se espere un usuario (identificarlo, mostrar su nombre, escribirle al correo) sin romper el significado.
2. **¿Bibliotecario es realmente un tipo de Usuario?** Sí, por el mismo motivo: es una persona registrada en el sistema con identificación, nombre y correo, y añade información particular (codigoEmpleado, turno). Matiz: la herencia supone que cada persona registrada es de un solo tipo. Si una misma persona pudiera ser estudiante y bibliotecario a la vez (p. ej. monitor), convendría modelar “roles” por composición. **Supuesto:** en este bloque cada usuario es de un único tipo.
3. **¿Bastan los atributos repetidos para crear la superclase?** No. Repetir identificación, nombre y correo es una pista, no una justificación. La superclase es válida porque (a) el dominio nombra el concepto Usuario (R12), (b) se cumple «es-un» y (c) hay sustituibilidad. Si solo hubiera atributos repetidos, bastaría con duplicarlos o con composición.
4. **Situación en la que sería conceptualmente débil.** Crear una superclase “Registro” o “Entidad” para Libro, Estudiante y Prestamo solo porque los tres tienen un identificador o un nombre; o hacer que Reserva herede de Prestamo porque ambos tienen estudiante y fecha. Una Reserva no «es un» Préstamo (no hay ejemplar entregado), y sustituir uno por otro rompería las reglas del dominio.
5. **Decisión.** Se considera válida la generalización: **Usuario** es una clase abstracta (nunca existe un “usuario genérico”) con **Estudiante** y **Bibliotecario** como especializaciones. Quedó reflejada en la Figura 1 y en el código (Sección 5.1).

Actividad 4 - Interfaces: expresar un contrato

4.1 Qué clases implementan Notificable

- **Estudiante: sí.** Debe recibir avisos de renovación, vencimiento y disponibilidad de una reserva (R13).
- **Bibliotecario: sí (supuesto).** Recibe avisos operativos, como ejemplares vencidos o reservas pendientes. Si el equipo decidiera que el bibliotecario no recibe mensajes del sistema, bastaría con quitarle la cláusula implements.
- **Usuario: no.** R13 dice “algunos usuarios”; si Usuario implementara Notificable, todos sus futuros subtipos quedarían obligados a hacerlo. La capacidad de ser notificado no forma parte de la definición «es-un» de usuario.
- **Libro, Ejemplar, Prestamo, Renovacion: no.** No son destinatarios de mensajes.

4.2 Representación en UML

En el diagrama UML número 1, Notificable aparece con el estereotipo «interface» y recibe una relación de realización (línea punteada con triángulo hueco) desde Estudiante y Bibliotecario. La generalización hacia Usuario (línea continua) coexiste con la realización en las mismas clases.

¿Qué garantiza Notificable y qué NO especifica? Garantiza la existencia de la operación `notificar(String)` con esa firma. No especifica el canal, el formato o contenido válido del mensaje, qué ocurre con un mensaje nulo, si la entrega es síncrona, si se almacena, ni cómo se manejan los fallos.

¿Por qué una interfaz no equivale a una superclase? Una superclase comparte estado y comportamiento y define identidad conceptual (es-un); una interfaz solo expresa una capacidad, no aporta atributos y puede cumplirla cualquier clase de cualquier jerarquía. Además, una clase puede implementar muchas interfaces, pero extender una sola clase.

4.3 Preguntas del taller

Pregunta	Respuesta del equipo
¿Qué promete una clase que implementa <code>Notificable</code> ?	Que cualquiera puede invocar <code>notificar(String mensaje)</code> sobre sus objetos, y que se les puede tratar como “algo notificable” sin conocer su clase concreta.
¿La interfaz define cómo se envía el mensaje?	No. Solo fija la firma. Estudiante hoy imprime en consola; mañana otra clase podría enviar un correo o un SMS sin cambiar el contrato.
¿Puede una clase heredar de <code>Usuario</code> e implementar <code>Notificable</code> al mismo tiempo?	Sí: <code>class Estudiante extends Usuario implements Notificable</code> . Java permite una sola superclase, pero varias interfaces. Expresan cosas distintas: <code>extends</code> dice qué ES; <code>implements</code> dice qué SABE HACER.

Actividad 5 – Implementación representativa en Java

Código fuente

Notificable.java

```
package smartlibrary;

/**
 * Contrato de capacidad: cualquier objeto que lo cumpla puede recibir
 * un mensaje del sistema.
 *
 * Garantiza: existe una operación notificar(String) que puede invocarse.
 * NO especifica: el canal (consola, correo, SMS), el formato, la
 * persistencia ni qué ocurre si la entrega falla.
 */
public interface Notificable {
    void notificar(String mensaje);
}
```

Usuario.java

```
package smartlibrary;

/**
 * Abstracción de toda persona registrada en SmartLibrary (R12).
 * Es abstracta: en el sistema no existe un "usuario genérico",
 * siempre es un estudiante o un bibliotecario.
 */
public abstract class Usuario {
    private final String identificacion;
    private final String nombre;
    private final String correo;

    protected Usuario(String identificacion, String nombre, String correo) {
        if (identificacion == null || identificacion.isBlank()) {
            throw new IllegalArgumentException("La identificación es obligatoria.");
        }
        if (nombre == null || nombre.isBlank()) {
            throw new IllegalArgumentException("El nombre es obligatorio.");
        }
        if (correo == null || !correo.contains("@")) {
            throw new IllegalArgumentException("El correo no es válido.");
        }
        this.identificacion = identificacion;
        this.nombre = nombre;
        this.correo = correo;
    }

    public String getIdentificacion() { return identificacion; }
    public String getNombre() { return nombre; }
    public String getCorreo() { return correo; }

    @Override
```

```

    public String toString() {
        return getClass().getSimpleName() + "[" + identificacion + " - " + nombre + "]";
    }
}

```

Estudiante.java

```

package smartlibrary;

```

```

/** Usuario que solicita préstamos y reservas. Puede recibir notificaciones (R13). */

```

```

public class Estudiante extends Usuario implements Notificable {
    private final String codigoEstudiantil;
    private final String programaAcademico;

    public Estudiante(String identificacion, String nombre, String correo,
        String codigoEstudiantil, String programaAcademico) {
        super(identificacion, nombre, correo);
        this.codigoEstudiantil = codigoEstudiantil;
        this.programaAcademico = programaAcademico;
    }

    public String getCodigoEstudiantil() { return codigoEstudiantil; }
    public String getProgramaAcademico() { return programaAcademico; }

    @Override
    public void notificar(String mensaje) {
        // Simulación del canal: consola.
        System.out.println("[NOTIFICACIÓN -> Estudiante " + getNombre()
            + " <" + getCorreo() + ">] " + mensaje);
    }
}

```

Bibliotecario.java

```

package smartlibrary;

```

```

/**
 * Usuario que administra la biblioteca.
 * Decisión: implementa Notificable porque recibe avisos operativos
 * (p. ej. ejemplares vencidos o reservas pendientes). Supuesto documentado.
 */

```

```

public class Bibliotecario extends Usuario implements Notificable {
    private final String codigoEmpleado;
    private final String turno;

    public Bibliotecario(String identificacion, String nombre, String correo,
        String codigoEmpleado, String turno) {
        super(identificacion, nombre, correo);
        this.codigoEmpleado = codigoEmpleado;
        this.turno = turno;
    }
}

```

```

    }

    public String getCodigoEmpleado() { return codigoEmpleado; }
    public String getTurno()      { return turno; }

    @Override
    public void notificar(String mensaje) {
        System.out.println("[NOTIFICACIÓN -> Bibliotecario " + getNombre()
            + " (turno " + turno + ")] " + mensaje);
    }
}

```

Libro.java

```

package smartlibrary;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * Título del catálogo. Es el TODO en la composición Libro ◆— Ejemplar:
 * es el único que crea sus ejemplares (R10).
 */
public class Libro {
    private final String isbn;
    private final String titulo;
    private final String autor;
    private final List<Ejemplar> ejemplares = new ArrayList<>();

    public Libro(String isbn, String titulo, String autor) {
        this.isbn = isbn;
        this.titulo = titulo;
        this.autor = autor;
    }

    /** Crea un ejemplar que nace ligado a este libro y no existe fuera de él. */
    public Ejemplar agregarEjemplar(String codigoInventario) {
        Ejemplar e = new Ejemplar(codigoInventario, this);
        ejemplares.add(e);
        return e;
    }

    public String getIsbn() { return isbn; }
    public String getTitulo() { return titulo; }
    public String getAutor() { return autor; }

    public List<Ejemplar> getEjemplares() {
        return Collections.unmodifiableList(ejemplares);
    }
}

```

```
}
```

Ejemplar.java

```
package smartlibrary;
```

```
/**
```

```
 * Copia física de un libro (R10). Constructor visible solo en el paquete
```

```
 * para que únicamente Libro pueda crearlo (refleja la composición).
```

```
 */
```

```
public class Ejemplar {
```

```
    private final String codigoInventario;
```

```
    private final Libro libro; // cada ejemplar corresponde a un único libro
```

```
    Ejemplar(String codigoInventario, Libro libro) {
```

```
        this.codigoInventario = codigoInventario;
```

```
        this.libro = libro;
```

```
    }
```

```
    public String getCodigoInventario() { return codigoInventario; }
```

```
    public Libro getLibro() { return libro; }
```

```
    @Override
```

```
    public String toString() {
```

```
        return "Ejemplar[" + codigoInventario + " de " + libro.getTitulo() + "];"
```

```
    }
```

```
}
```

Renovacion.java

```
package smartlibrary;
```

```
import java.time.LocalDate;
```

```
/**
```

```
 * Registro histórico e inmutable de una renovación (R11).
```

```
 * Es PARTE de Prestamo: solo Prestamo puede crearla (constructor de paquete).
```

```
 */
```

```
public class Renovacion {
```

```
    private final LocalDate fechaRenovacion; // cuándo se realizó
```

```
    private final LocalDate fechaAnterior; // fecha de devolución previa
```

```
    private final LocalDate nuevaFecha; // nueva fecha de devolución
```

```
    Renovacion(LocalDate fechaRenovacion, LocalDate fechaAnterior, LocalDate nuevaFecha) {
```

```
        this.fechaRenovacion = fechaRenovacion;
```

```
        this.fechaAnterior = fechaAnterior;
```

```
        this.nuevaFecha = nuevaFecha;
```

```
    }
```

```
    public LocalDate getFechaRenovacion() { return fechaRenovacion; }
```

```

public LocalDate getFechaAnterior() { return fechaAnterior; }
public LocalDate getNuevaFecha() { return nuevaFecha; }

@Override
public String toString() {
    return "Renovacion[hecha el " + fechaRenovacion + ": " + fechaAnterior
        + " -> " + nuevaFecha + "]";
}
}

```

Prestamo.java

```

package smartlibrary;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * Préstamo de un ejemplar a un estudiante.
 * - Asociación con Estudiante y con Ejemplar (existen por sí mismos).
 * - Composición con Renovacion (el historial pertenece al préstamo).
 */
public class Prestamo {
    private final Estudiante estudiante;
    private final Ejemplar ejemplar;
    private final LocalDate fechaPrestamo;
    private LocalDate fechaPrevistaDevolucion;
    private final List<Renovacion> renovaciones = new ArrayList<>();

    public Prestamo(Estudiante estudiante, Ejemplar ejemplar,
        LocalDate fechaPrestamo, LocalDate fechaPrevistaDevolucion) {
        if (estudiante == null || ejemplar == null) {
            throw new IllegalArgumentException("Estudiante y ejemplar son obligatorios.");
        }
        if (!fechaPrevistaDevolucion.isAfter(fechaPrestamo)) {
            throw new IllegalArgumentException(
                "La fecha prevista debe ser posterior a la fecha del préstamo.");
        }
        this.estudiante = estudiante;
        this.ejemplar = ejemplar;
        this.fechaPrestamo = fechaPrestamo;
        this.fechaPrevistaDevolucion = fechaPrevistaDevolucion;
    }

    /** Regla protegida: la nueva fecha debe ser estrictamente posterior a la vigente. */
    public void renovar(LocalDate nuevaFecha) {
        renovar(nuevaFecha, LocalDate.now());
    }
}

```

```

/** Variante con fecha de renovación explícita (útil para pruebas reproducibles). */
public void renovar(LocalDate nuevaFecha, LocalDate fechaRenovacion) {
    if (nuevaFecha == null) {
        throw new IllegalArgumentException("La nueva fecha es obligatoria.");
    }
    // 1. validar
    if (!nuevaFecha.isAfter(fechaPrevistaDevolucion)) {
        throw new IllegalArgumentException("Renovación inválida: la nueva fecha ("
            + nuevaFecha + ") debe ser posterior a la fecha prevista vigente ("
            + fechaPrevistaDevolucion + ").");
    }
    // 2. crear 3. almacenar
    renovaciones.add(new Renovacion(fechaRenovacion, fechaPrevistaDevolucion, nuevaFecha));
    // 4. actualizar
    fechaPrevistaDevolucion = nuevaFecha;
}

public Estudiante getEstudiante()      { return estudiante; }
public Ejemplar getEjemplar()          { return ejemplar; }
public LocalDate getFechaPrestamo()     { return fechaPrestamo; }
public LocalDate getFechaPrevistaDevolucion() { return fechaPrevistaDevolucion; }
public int getCantidadRenovaciones()    { return renovaciones.size(); }

/** Vista de solo lectura: nadie externo puede alterar el historial. */
public List<Renovacion> getRenovaciones() {
    return Collections.unmodifiableList(renovaciones);
}
}

```

Main.java

```

package smartlibrary;

import java.time.LocalDate;

/** Prueba mínima: una renovación válida y una inválida. */
public class Main {
    public static void main(String[] args) {
        Estudiante estudiante = new Estudiante("1085", "Ana Torres",
            "ana.torres@udenar.edu.co", "EST-2024-017", "Ingeniería de Sistemas");
        Bibliotecario bibliotecario = new Bibliotecario("2090", "Luis Mora",
            "luis.mora@smartlibrary.edu", "EMP-005", "Mañana");

        Libro libro = new Libro("978-0132350884", "Clean Code", "Robert C. Martin");
        Ejemplar ejemplar = libro.agregarEjemplar("EJ-001");
        libro.agregarEjemplar("EJ-002");

        Prestamo prestamo = new Prestamo(estudiante, ejemplar,
            LocalDate.of(2026, 10, 1), LocalDate.of(2026, 10, 8));
    }
}

```

```

System.out.println("=== Datos iniciales ===");
System.out.println(estudiante + " | " + bibliotecario);
System.out.println(libro.getTitulo() + " tiene " + libro.getEjemplares().size() + " ejemplares");
System.out.println("Fecha prevista inicial: " + prestamo.getFechaPrevistaDevolucion());

// ---- PRUEBA 1: renovación válida ----
System.out.println("\n=== Prueba 1: renovación válida ===");
prestamo.renovar(LocalDate.of(2026, 10, 15), LocalDate.of(2026, 10, 7));
estudiante.notificar("Su préstamo fue renovado.");
System.out.println("Nueva fecha prevista: " + prestamo.getFechaPrevistaDevolucion());
System.out.println("Cantidad de renovaciones: " + prestamo.getCantidadRenovaciones());
prestamo.getRenovaciones().forEach(System.out::println);

// ---- PRUEBA 2: renovación inválida ----
System.out.println("\n=== Prueba 2: renovación inválida (fecha igual a la vigente) ===");
try {
    prestamo.renovar(LocalDate.of(2026, 10, 15), LocalDate.of(2026, 10, 14));
    System.out.println("ERROR: debió rechazarse.");
} catch (IllegalArgumentException ex) {
    System.out.println("Rechazada correctamente -> " + ex.getMessage());
}

System.out.println("\n=== Prueba 2b: renovación inválida (fecha anterior) ===");
try {
    prestamo.renovar(LocalDate.of(2026, 10, 10), LocalDate.of(2026, 10, 14));
    System.out.println("ERROR: debió rechazarse.");
} catch (IllegalArgumentException ex) {
    System.out.println("Rechazada correctamente -> " + ex.getMessage());
}

System.out.println("\nEstado final (sin cambios por las inválidas):");
System.out.println("Fecha prevista: " + prestamo.getFechaPrevistaDevolucion());
System.out.println("Cantidad de renovaciones: " + prestamo.getCantidadRenovaciones());

// ---- Polimorfismo por contrato ----
System.out.println("\n=== Polimorfismo mediante Notificable ===");
Notificable[] destinatarios = { estudiante, bibliotecario };
for (Notificable n : destinatarios) {
    n.notificar("Recordatorio general del sistema.");
}
}
}

```

Evidencia de ejecución

Comandos usados:

```
javac -encoding UTF-8 -d out src/smartlibrary/*.java  
java -cp out smartlibrary.Main
```

Salida obtenida:

=== SMARTLIBRARY - BLOQUE 5 ===

Estudiante: Laura Gomez

Bibliotecario: Carlos Ruiz

Ejemplar disponible al prestar: false

Renovacion valida: OK

Nueva fecha prevista: 2026-10-15

Cantidad de renovaciones: 1

Notificacion para Laura Gomez: Su prestamo fue renovado.

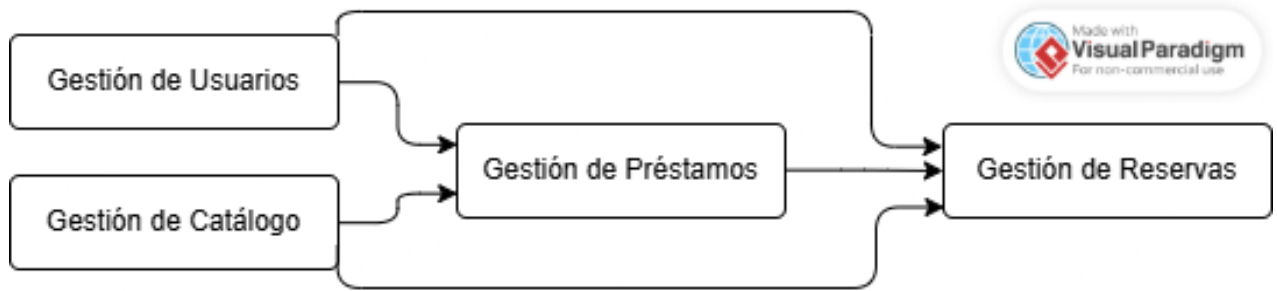
Renovacion invalida: rechazada correctamente.

Motivo: La nueva fecha debe ser posterior a la fecha prevista vigente.

Qué debe ocurrir con una renovación inválida (documentado): Prestamo.renovar lanza una IllegalArgumentException cuando la nueva fecha es igual o anterior a la fecha prevista vigente. La excepción se produce **antes** de crear la Renovacion, así que el préstamo conserva su fecha prevista y su historial queda intacto. En la salida se ve: la fecha sigue en 2026-10-15 y la cantidad de renovaciones sigue en 1 después de los dos intentos inválidos (fecha igual y fecha anterior).

Vista funcional mediante componentes UML

Componente	Clases relacionadas	¿De qué otro componente necesita información? ¿Para qué?
Gestión de Usuarios	Usuario, Estudiante, Bibliotecario, Notificable	De ninguno: es el origen de la identidad y de la capacidad de notificación que usan los demás.
Gestión de Catálogo	Libro, Ejemplar	De ninguno: es la fuente de títulos y ejemplares.
Gestión de Préstamos	Prestamo, Renovacion	Usuarios: identificar al estudiante y notificarle la renovación. Catálogo: conocer el ejemplar que se presta.
Gestión de Reservas	Reserva	Usuarios: saber quién reserva. Catálogo: saber qué libro se reserva. Préstamos: saber si los ejemplares del libro están prestados.



Diferencia entre una clase y un componente. Una clase del diagrama define estructura y comportamiento de un tipo de objeto; un componente agrupa clases que cumplen una responsabilidad funcional y muestra qué necesita de otras partes. Esta vista es de alto nivel: no incluye microservicios, infraestructura, contenedores, despliegue, APIs ni bases de datos.

Conclusión

«¿Qué cambia en el diseño cuando dejamos de mirar clases aisladas y empezamos a diseñar sus relaciones y contratos?»

Al dejar de mirar clases aisladas, el diseño pasa de ser una lista de atributos a expresar reglas del dominio. Las relaciones indican quién conoce a quién, quién controla el ciclo de vida de quién (Prestamo con sus Renovaciones, Libro con sus Ejemplares) y con qué multiplicidad. La herencia exige comprobar que Estudiante y Bibliotecario realmente «son» Usuarios, no solo que comparten atributos. La interfaz Notificable separa lo que un objeto promete hacer de la forma en que lo hace, y permite tratar igual a objetos de jerarquías distintas. Además, el código empieza a proteger esas reglas: Prestamo valida sus renovaciones y solo Libro crea Ejemplares. Cada decisión se apoya en un requisito, lo que da trazabilidad y prepara la evaluación de calidad del siguiente bloque.